Spline Trajectory for Three Wheel Mobile Robot Swerve Drive

Prishandy Hamami Amrulloh
Electrical Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
prishandyhamami@gmail.com

Ali Husein Alasiry
Electrical Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
ali@pens.ac.id

Eko Henfri Binugroho
Mechatronic Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
sragen@pens.ac.id

Ardik Wijayanto
Electrical Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
ardik@pens.ac.id

Mawaddah Sekar Rahmawati
Electrical Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya, Indonesia
mawaddahsekarr@gmail.com

Himmawan Sabda Maulana
Mechatronic Engineering Department
Politeknik Elektronika Negeri Surabaya
Surabaya Indonesia
himmawan@pens.ac.id

2023 International Electronics Symposium (IES) | 979-8-3503-1473-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/IES59143.2023.10242512

***Abstract*— Wheeled mobile robot platform has been widely used in various field. Some mobile platforms support holonomic motion and others only support non-holonomic motion. A mobile platform that supports holonomic motion became more popular recently since it provides more flexibility in omnidirectional motion. Currently, Omni-wheels and mecanum-wheels based mobile robot platforms are the most common holonomic platforms in the market. They are simpler and cheaper to be implemented. But in applications, it needs more traction and higher efficiency, the swerve drive mobile robot platform will become a more viable solution. It has more smooth movement compared to other holonomic mobile robots because this type of mobile robot can rotate the wheel angle independently, so it has very little wheel slip against the floor. However, when the swerve drive mobile robot is run on an angled trajectory at high speed, it will experience steering delay so that the robot will come out of the predetermined track. Therefore, it is necessary to modify the angled trajectory into a curved line trajectory using the Spline method. On the Spline trajectory, swerve drive can run with higher speed and higher accuracy than the angled trajectory. On the bézier spline swerve drive can run with a velocity of up to 3.25 m/s with an average position error decrease of 36 mm on the bézier spline with ω=1, 72 mm on the bézier spline with various ω values. While on cubic spline it can run with a velocity up to 2.44 m/s with an average position error decrease of 69.25 mm.**

Keywords*— Mobile robot, Swerve drive, Trajectory, Spline, Bézier, Cubic

I. Introduction

Currently, the development of mobile robot technology is expanding. Many modifications in mechanical, electronic, and control design have been done to get an efficient, fast, accurate, and precise mobile robot [1]. One of the applications of mobile robot can be seen in an annual education and entertainment program namely ABU ROBOCON that hosted by Asia Pacific Broadcasting Union [2]. In this competition, robots must complete missions according to the theme rules that changed in every year. But in all theme, the most common problem lies in the mobile robot platform. During the competition, mobile robot needs to moves on a specific trajectory manually or automatically. In many cases, mobile robot must run quickly and accurately according to the specific target position, and the automatic trajectory control will have upper hand in this task compared to the manually driven robot [3-5].

As the mobile robot need to maneuver swiftly, there are many chances that acceleration and deceleration happen during its movement. Better traction between the robot's wheel and the floor benefits the robots as it will become easier to control during its swift maneuver. To address this problem, swerve drive type became one of the best models and most widely used in mobile robot platform [6-7]. This mobile robot has better capabilities in a curved trajectory than other holonomic types of mobile robots since it can adjust the motor speed and steering angle of each wheel independently. Therefore, this type of mobile robot has better maneuverability and speed compared to DDMR, Ackerman steering, and Omni wheel [8-10].

The swerve drive mobile robot has one weakness when the robot runs on an angled trajectory, there will be inertia and the steering delay in the wheels thus the robot will come out of the predetermined track. This problem can be overcome by modifying the line into a curved line. The spline equation is one of the methods used to bend the curve from the angled lines. Several spline methods are available to create curved lines, such as the Bézier Spline and the Cubic Spline [11-14]. Both spline types have different characteristics of the resulting curved lines and the parameters used, and will be implemented observed in this application.

II. Mobile Robot Swerve Drive

A. Holonomic

Generally, mobile robots are divided into two parts, non-holonomic and holonomic. Holonomic mobile robots have more maneuverability than non-holonomic mobile robots, robot as it has less motion constraint, as shown in Fig. 1.

Mobile robots that have non-holonomic mechanism such as Ackerman steering and DDMR have been widely used in applications that do not need omni-directional maneuver. Holonomic mobile robots including the omni-wheels and mecanum-wheels, and swerve drives open the ability for the mobile platform to move in any direction, thus the maneuver became more flexible.

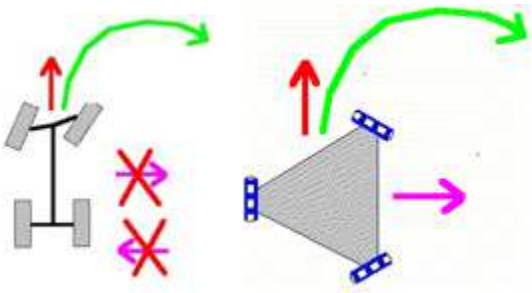


Fig. 1. Mobile Robot (a) non-holonomic (b) holonomic

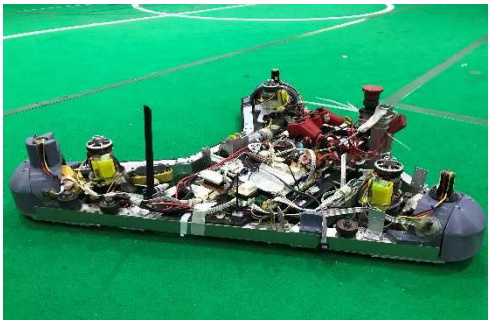


Fig 2. Platform mobile robot swerve drive

B. *Swerve Drive*

There are various configurations of the number of wheels on the Swerve Drive mobile robot ranging from three to four or more. This research used the developed three wheels swerve drive shown in Fig.2 became the test-bed to implement the spline trajectory. Our previous research entitled Position and Orientation Control of Three Wheels Swerve Drive Mobile Robot Platform results in kinematic equations of the swerve drive, as shown in (1) to (10) [6].

From Fig. 3, we can calculate the resultant vectors of the robot ω.r and VR for each wheel at any steering angle as V1, V2, and V3. VR denotes the robot speed as the resultant of the vectors Vx and Vy. Since Vx is perpendicular to Vy, hence VR can be determined as:

$$VR = \sqrt{Vx^2 + Vy^2} \tag{1}$$

For starting, the wheel speed vector is calculated as (2) to (4) by the assumption that the zero-steering angle of each wheel is in line with Vx.

Wheel 1:

$$\begin{aligned} V1x &= \omega.r + Vx \\ V1y &= Vy \end{aligned} \tag{2}$$

Wheel 2:

$$\begin{aligned} V2x &= -\omega.r.\sin(30°) + Vx \\ V2y &= -\omega.r.sin(60°) + Vy \end{aligned} \tag{3}$$

Wheel 3:

$$\begin{aligned} V2x &= -\omega.r.\sin(30°) + Vx \\ V2y &= \omega.r.sin(60°) + Vy \end{aligned} \tag{4}$$

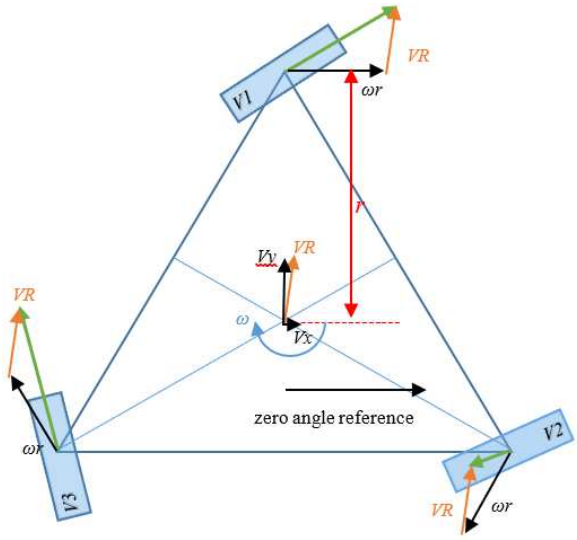


Fig. 3. Vector Ilustration of swerve drive

The robot can be moved in a planar motion in each desired speed as Vx , Vy, and ω. For this purpose, the inverse kinematics is needed. Equation (5) to (10) shows the inverse kinematics of each wheel as the resultant vector of each component in (2) to (4).

written as:

Wheel 1:

$$V1 = \sqrt{V1x^2 + V1y^2} \tag{5}$$

$$\theta 1 = atan2(V1x, V1y) \tag{6}$$

Wheel 2:

$$V2 = \sqrt{V2x^2 + V2y^2} \tag{7}$$

$$\theta 2 = atan2(V2x, V2y) \tag{8}$$

Wheel 3:

$$V3 = \sqrt{V3x^2 + V3y^2} \tag{9}$$

$$\theta 3 = atan2(V3x, V3y) \tag{10}$$

Where:

ω	= rotation speed of the robot through its center body
r	= distance of the wheel from the center body
VR	= The mobile robot's translation speed vector
Vx	= translation speed vector in the x-axis (body-fixed)
Vy	= translation speed vector in the y-axis (body-fixed)
$V1$	= translation speed of the wheel 1
$V2$	= translation speed of the wheel 2
$V3$	= translation speed of the wheel 3

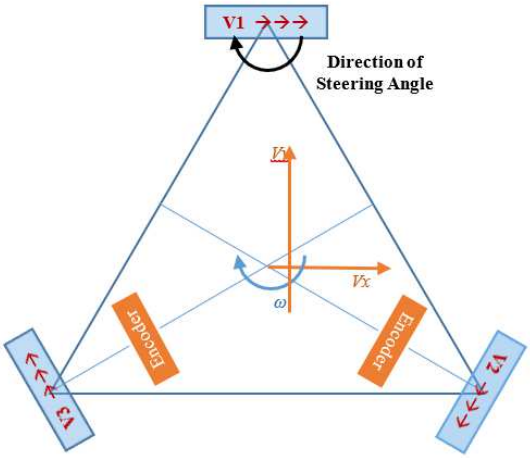


Fig. 4. Displacement of the encoders and robot's zero orientation

Using position sensors that consist of IMU for the rotation and rotary encoder for translation, then the mobile robot orientation and position can be tracked [7]. Using this data as the feedback, the mobile robot thus can be controlled using a PID controller calculated based on errors calculated from equations (15) to (17). Fig. 4. shows the angle of each wheel to the robot body and the location of the rotary encoder sensor as a position sensor. The current robot's speed Vx and Vy are obtained from the speed data of encoder one (Ev1) and speed of encoder two (Ev2) which can be written as:

$$V_x = \left(\frac{-(Ev1 + Ev2)}{2}\right).sin(30°) \quad (11)$$

$$V_y = \left(\frac{(Ev1 - Ev2)}{2}\right).sin(60°) \quad (12)$$

The robot's heading α to the real world is obtained through the IMU orientation sensor. In this study use the IMU sensor type MPU6050. In (13) to (14) α is used to transform the *Vy* and *Vx* into the real-world coordinate as *Sy* and *Sx* as follows:

$$S_x = V_x \cos(\alpha) + V_y \sin(\alpha) \quad (13)$$

$$S_y = -V_x \sin(\alpha) + V_y \cos(\alpha) \quad (14)$$

The position of the current robot in real-world coordinates is determined by integrating each *Sx* and *Sy* through time. The current robot's position is *Px* and *Py*, while the target position is *Tx* and *Ty*. The goal is to calculate the error distance to a given target, as in (15) to (17).

$$Error_x = Tx - Px \quad (15)$$

$$Error_y = Ty - Py \quad (16)$$

$$Distance_{Error} = \sqrt{\left(Error_x{}^2 + Error_y{}^2\right)} \quad (17)$$

Distance error denoted as $Distance_{Error}$ is used on the PID controller to calculate the translation speed of the robot (*VR*). Inverse kinematics requires data on x-axis velocity *Vx* and the y-axis velocity *Vy*. *Vx* and *Vy* are calculated using *VR* times the angle β calculated from error of the robot position on the x-axis and the y-axis, as shown in (18) to (20).

$$\beta = atan2\ [Error_x, Error_Y] \quad (18)$$

$$Vx = VR.sin(\beta) \quad (19)$$

$$Vy = VR.cos(\beta) \quad (20)$$

Position control is calculated using (11) to (20), while robot orientation control is calculated by the difference in readings from the IMU MPU6050 sensor and the given target heading using PID control.

III. Spline Trajectory Controller

A. Spline Trajectory

In trajectory control using Spline, the target points that form an angle will be modified into curved lines. The modification of the trajectory can be done using two types, namely the Bézier Spline and the Cubic Spline. The two types differ in the characteristics and parameters used. Both equations must be adjusted to the needs of the desired trajectory. Making a robot trajectory from the splines equation resulting in target position, namely *Tx* and *Ty*, in (15) and (16).

B. Bézier Spline

In the Bézier Spline, the modified target point position will always be within the input reference point. The trajectory modification using the Bézier Spline can be calculated using (21) based on the previously inputted spline control points notated as P_i.

$$B(t) = \frac{\sum_{i=0}^{n}\binom{n}{i}t^i(1-t)^{n-i}P_i\omega_i}{\sum_{i=0}^{n}\binom{n}{i}t^i(1-t)^{n-i}\omega_i} \quad (21)$$

Where:

$B(t)$	=	Bézier Spline function
n	=	The number of references
i	=	Iteration *(i=0,1,2,…,n)*
t	=	Spline point counter value
P	=	References point
ω_i	=	Spline curvature value

To form a coordinate target point, least two axis variables are needed, namely the x-axis and the y-axis. The target point coordinates of the x-axis and y-axis are calculated separately, namely $B(t)_x$ for the x-axis and $B(t)_y$ for the y-axis. The constant t is the counter value where the smaller the value closer the distance between the Bézier Spline points. In addition, the parameter that is no less important is ω_i. These parameters will set the degree of curvature of the spline trajectory, as shown in Fig.5.

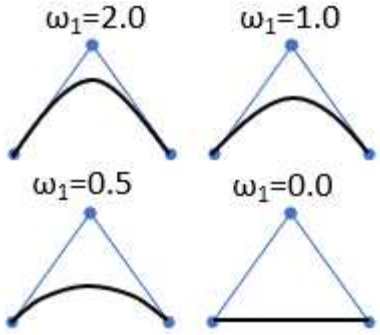


Fig. 5. The effect of ω_i to the Bézier curve

C. Cubic Spline

In the Cubic Spline, the curved line will pass through the input reference points. The inputted parameters are the coordinates of the x-axis, y-axis, and the number of desired output points. There is no line curvature controller to adjust the curvature in the Cubic Spline. A dot can be calculated using the general equation, as written by (22).

$$q_i = (1-t)y_{i-1} + ty_i + t(1-t)((1-t)a_i + tb_i) \quad (22)$$

Where:

q_i	=	cubic spline point
t	=	third-degree polynomials
i	=	Iteration *(i=0,1,2,…,n)*
a_i *and* b_i	=	cubic spline curvature
y_i	=	References point in y axis

The iteration *i* and t-coefficient in (23) is a third-degree polynomial interpolating *y*. (24) and (25) define the value of a_i and b_i.

$$t = \frac{x - x_{i-1}}{x_i - x_{i-1}} \quad (23)$$

$$a_i = k_{i-1}(x_i - x_{i-1}) - (y_i - y_{i-1}) \quad (24)$$

$$b_i = -k_i(x_i - x_{i-1}) - (y_i - y_{i-1}) \quad (25)$$

While the coefficient of k_i in (24) and (25) is the value of the curvature, it is calculated continuously based on the previous point value. However, it should be noted that the end

of the curved line will experience discontinuity, whether at the beginning of the line or the end of the line. Thus, there are differences in equations for finding the values of k_0, k_i, and k_n, as shown by (26) to (28).

$$k_0 = 3\frac{y_1 - y_0}{(x_1 - x_0)^2} \tag{26}$$

$$k_i = 3\left(\frac{y_i - y_{i-1}}{(x_i - x_{i-1})^2} + \frac{y_{i+1} - y_i}{(x_{i+1} - x_i)^2}\right) \tag{27}$$

$$k_n = 3\frac{y_n - y_{n-1}}{(x_n - x_{n-1})^2} \tag{28}$$

Where:

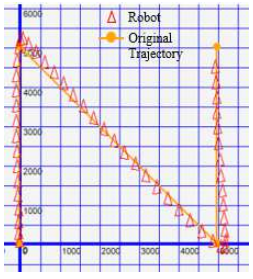
k_0	=	First point curvature value
k_i	=	The i-th index point curvature value
k_n	=	Last point curvature value
y_0	=	0^{th} y point
y_1	=	1^{st} y point
x_0	=	0^{th} x point
x_1	=	1^{st} x point
y_i	=	Point value y for the i-th index

IV. Experiment Results

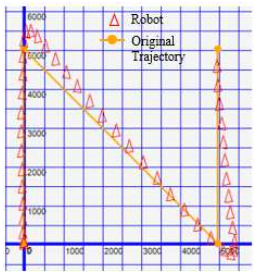
In this experiment, the mobile robot is run on various forms of angled trajectory and modified trajectory using a spline. The mobile robot headings are set to the same orientation, which is 0°. The robot is controlled using a PID controller to reach the inputted target point or the target point created using a spline. This control method can be implemented easily on microcontroller used in the robot [15]. The position and orientation data of the mobile robot are sent to the computer and plotted using a GUI application. Data is sampled every 25ms for the original trajectory, while the modified trajectory data is sent when the mobile robot reaches the finish point. In the first experiment, position error refers to the difference between the mobile robot position and the angled trajectory for the non-spline method. For the second experiment, we compare both spline methods, thus position error refers to the difference between the mobile robot position and Bézier or Cubic Spline trajectory.

A. Original Trajectory

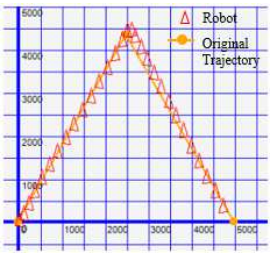
The tests were carried out at different turning angles representing 45°, 60°, 90°, and 120°. The constant speed applied in this test is at 2.44 m/s and 3.25 m/s. Fig. 6 shows the results of the mobile robot's trajectory to follow the angled trajectory without using a spline at several turning angles



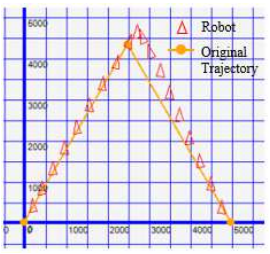
a. Turning angle at 45° and Speed is limited at 2,44 m/s



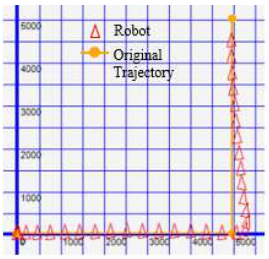
b. Turning angle at 45° and Speed is limited at 3,25 m/s



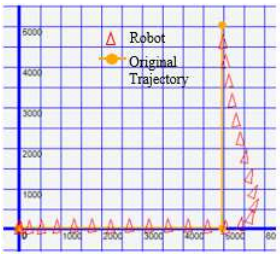
c. Turning angle at 60° and Speed is limited at 2,44 m/s



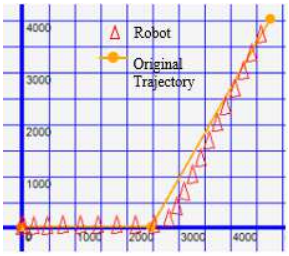
d. Turning angle at 60° and Speed is limited at 3,25 m/s



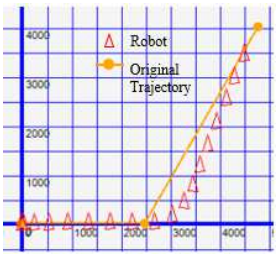
e. Turning angle at 90° and Speed is limited at 2,44 m/s



f. Turning angle at 90° and Speed is limited at 3,25 m/s



g. Turning angle at 120° and Speed is limited at 2,44 m/s



h. Turning angle at 120° and Speed is limited at 3,25 m/s

Fig. 6. Original Trajectory with various turning angle

The orange line with the orange dot indicates the path that the swerve drive must follow. The red triangle shows the mobile robot's position and orientation during its movement. Based on Fig. 6, in the corner of each trajectory the robot experiences and an out of track as shown by the mobile robot's position is far from the predetermined trajectory. The error value at various turning angles can be seen in Table I.

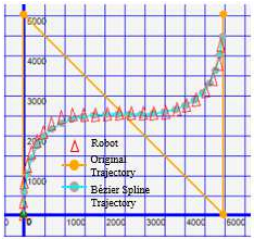
TABLE I. Position Error for Trajectory Without Spline

No.	Turning Angle (°)	Position Error (mm)	
		Limited speed 2,44 m/s	Limited speed 3,25 m/s
1	45	211	448
2	60	101	368
3	90	330	745
4	120	249	482

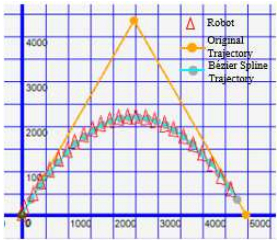
The positional error value is a bit high at each turn angle. It causes the mobile robot to go out of a predetermined trajectory and require more time to reach a finish point. The greater the value of the speed, the greater the position error. So, increasing the limit speed value cannot shorten the travel time to reach the finish point.

B. Bézier Spline Trayectory with ω=1

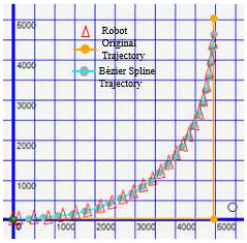
In the testing trajectory using the Bézier Spline with ω = 1, the mobile robot runs at 3.25 m/s with various Bézier Spline trajectories. The t value given to the spline parameter is 0.04. The blue line with a gray dot represents the Bézier Spline trajectory. The red triangle shows the mobile robot's position and orientation during its movement, while the orange line represents the original trajectory as a reference for the spline calculation.



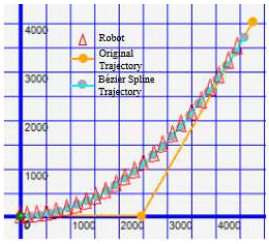
a. Turning angle at 45



b. Turning angle at 60



c. Turning angle at 90



d. Turning angle at 120

Fig. 7. Bézier Spline Trajectory with various turning angle and ω=1

Fig. 7 shows that the swerve drive runs on the Bézier Spline trajectories in various turning angles well. Bézier Spline trajectory can reduce the positional error value of the mobile robot swerve drive, as shown in Table II.

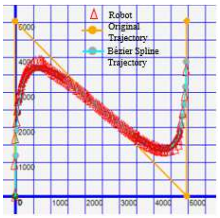
TABLE II. POSITION ERROR IN BÉZIER SPLINE TRAJECTORY WITH ω=1

No.	Turning Angle (°)	Position Error (mm)
1	45	19
2	60	14
3	90	46
4	120	65

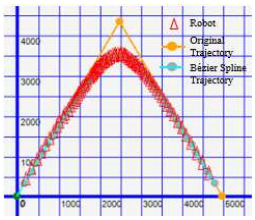
Although the Bézier Spline trajectory with ω=1 error value in various turning angles is smaller than the position error value on the line without using a spline. The trajectory generated using Bézier Spline with ω=1 result in a track that far from the given input reference point. To overcome this, it needs to increase the ω parameter value of the Bézier Spline equation.

C. *Bézier Spline Trajectory with various ω value*

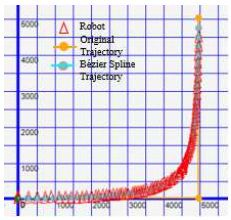
In this trajectory test using the Bézier Spline, the mobile robot runs at the same speed limit of 3.25 m/s with various values of ω and the turning angle. T equals 0.01. It is smaller than the test before because the Bézier Spline requires more points to form a sharper curve. Fig. 8 shows the results of the Bézier Spline trajectory with various ω values. With the addition of the ω value to the Bézier Spline parameter, the resulting path is closer to the input reference point (orange line).



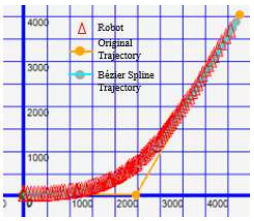
a. Turning angle at 45° and ω=10



b. Turning angle at 60° and ω=4



c. Turning angle at 90° and ω=3



d. Turning angle at 120° and ω=2

Fig. 8. Bézier Spline Trajectory with various turning angle and ω value

When the trajectory angle is smaller, the calculation requires a higher ω value to approach closer to the reference point. Table III shows the error position generated in the Bézier trajectory with various ω values.

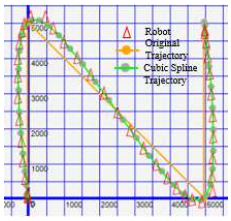
TABLE III. POSITION ERROR IN BÉZIER SPLINE TRAJECTORY WITH VARIOUS ω

No.	Turning Angle (°)	ω	Position Error (mm)
1	45	10	167
2	60	4	34
3	90	3	51
4	120	2	36

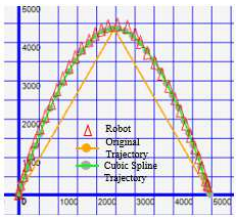
Based on Table III, the positional error value when the robot tracks the trajectory generated using spline is smaller than the one without using a spline. The 45° turn angle has the highest positional error because it uses a higher ω value. With a higher ω value, the resulting trajectory will resemble an angled line.

D. *Cubic Spline*

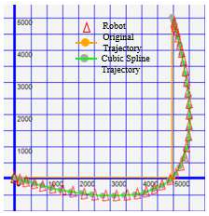
In testing the trajectory control from the trajectories generated using Cubic Spline, the mobile robot is set to run at the speed limit of 2.44 m/s with various Cubic Spline trajectories. The green line with a gray dot represents the Cubic spline trajectory. The red triangle shows the mobile robot's position and orientation during its movement, while the orange line represents the original trajectory as a reference for the spline calculation. The results of this experiment are shown in Fig. 9.



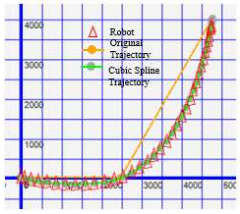
a. Turning angle at 45°



b. Turning angle at 60°



c. Turning angle at 90°



d. Turning angle at 120°

Fig. 9. Cubic Spline Trajectory with various turning angle

In Cubic Spline trajectories, the reference point always be passed by the modified spline point. As a result, the cubic spline trajectory has a longer distance. However, with a small positional error value, the travel time using the Cubic Spline is still the same as the trajectory without using a spline, but the advantage is that the mobile robot's movement is more estimated because there is no mobile robot movement out of

its line. The value of the position error on the Cubic Spline trajectory can be seen in Table IV.

TABLE IV. POSITION ERROR IN CUBIC SPLINE TERAJECTORY

No.	Turning Angle (°)	Position Error (mm)
1	45	152
2	60	68
3	90	41
4	120	16

Table IV shows that the positional error value on the Cubic Spline trajectory is smaller than the trajectory without using a spline. A comparison of position errors from every test in different angles and trajectory types is shown in clustered column chart in Fig. 10.

The lowest position error occurs when the mobile robot runs on a spline trajectory. Which is on the Bézier Spline trajectory with ω=1 (column with red dots), with various ω value (column with black diamond line), and on the cubic trajectory spline (column with green horizontal lines). It has almost the same error turning angle at 60°, 90°, and 120°. However, the cubic Spline trajectory is not better than Bézier because the mobile robot runs at a slower velocity of 2.44 m/s, while in Bézier Spline the mobile robot runs at 3.25 m/s. Columns with blue vertical lines show that the mobile robot running at 3.25 m/s without a spline has the highest positional error, especially at a 90° turning angle. When the mobile robot runs at a slower velocity of 2.44 m/s (column with yellow diagonal line), the position error becomes smaller but still higher than when the mobile robot runs on spline trajectories.

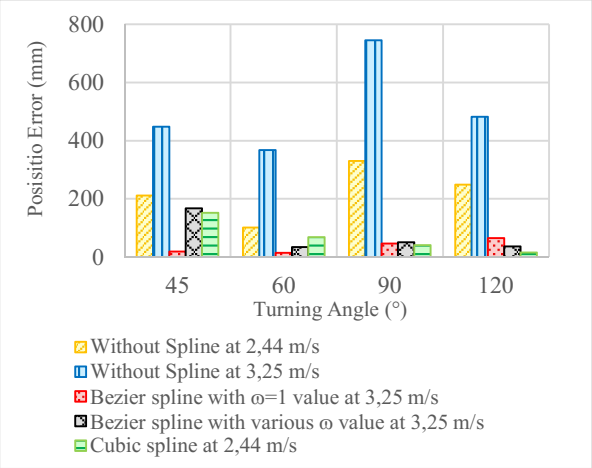


Fig. 10. The comparisons of position Error

V. CONCLUSION

By modifying the swerve drive mobile robot trajectory using from a hard turn to the spline equation, the mobile robot runs smoother with average position error decrease of 36 mm on the Bézier Spline with ω=1, 72 mm on the Bézier Spline with various ω value, and 69.25 mm on the Cubic Spline. But these solutions are without caveats, as the generated trajectories smoothing the robot paths in to some degree, thus the selection of which method to generate the trajectory will be depend on the application. For simplicity the angled turn will solve the problem by adjusting the robot speed. When the given points are not needed to be passed by the robot, the Bézier Spline can be chosen with the smoothing effect of the generated trajectory can be adjusted easily by changing the ω value. Cubic Spline can be chosen instead when the given points should be passed by the robot with faster robot's movement, but it tends to generate longer path than other methods.

ACKNOWLEDGMENT

This research is supported by EIRA robotics team at Politeknik Elektronika Negeri Surabaya (PENS).

REFERENCES

[1] Záhorþák, Gabriel; Petráš, Ivo, "Control system of mobile robot," *International Carpathian Control Conference (ICCC),* no. 12, pp. 469-473, 2011.

[2] "ABU Robocon 2022 Theme & Rules," [Online]. Available: https://www.aburobocon2022.com/index.php/game-rules. [Diakses 8 Agustus 2022].

[3] Li, Weihao; Yang, Chenguang; Jiang, Yiming; Liu, Xiaofeng; Su, Chun-Yi, "Motion Planning for Omnidirectional Wheeled Mobile Robot by," *Journal of Advanced Transportation,* pp. 1-11, 2017.

[4] Gavani, Madhukar; Tanpure, Dhananjay; Falake, Parasram, "Path Planning of Three Wheeled Omni-Directional Robot Using Bezier Curve Tracing Technique and PID control Algorithm," *Pune Section International Conference (PuneCon),* 2019.

[5] Abidin, Zainul; Muis, Muhram; Djuriatno, Waru, "Omni-Wheeled Robot with Rapidly-exploring Random Tree (RRT) Algorithm for Path Planning," *International Conference on Advanced Mechatronics, Intelligent Manufacture and Industrial Automation (ICAMIMIA),* 2019.

[6] E. H. Binugroho, A. Setiawan, Y. Sadewa, P. H. Amrulloh, K. Paramasastra dan R. W. Sudibyo, "Position and Orientation Control of Three Wheels," *International Electronics Symposium (IES),* 2021.

[7] Y. Sadewa, E. H. Binugroho, N. Hanafi, I. D. Pramadihanto, A. Fauzi dan A. Purwanto, "Wall Following and Obstacle Avoidance Control in Roisc-v1.0 (Robotic Disinfectant) using Behavior Based Control," *International Electronics Symposium (IES),* 2021.

[8] R. Zhang, H. Hu dan Y. Fu, "Trajectory tracking for omnidirectional mecanum robot with longitudinal slipping," *MATEC Web of Conferences 256(7):02003,* 2019.

[9] Mamun, Md. Abdullah Al; Nasir, Mohammad Tariq; Khayyat, Ahmad, "Embedded System for Motion Control of an Omnidirectional Mobile Robot," *JOURNAL OF LATEX CLASS FILES,* vol. XIV, pp. 6722 - 6739, 2015.

[10] Ijaabo, Enas M.; Alsharkawi, Adham; Firdaus, Ahmad Riyad, "Trajectory Tracking of an Omnidirectional Mobile," *International Conference on Applied Engineering (ICAE),* 2019.

[11] S. A. Dyer dan J. S. Dyer, "Cubic-Spline Interpolation," *IEEE,* pp. 44-46, 2001.

[12] M. F. Alfatih, M. A. Riyadi dan I. Setiawan, "Speed control system in mobile robot based on Bezier curve," *International Conference on Innovation in Science and Technology,* pp. 1-10, 2019.

[13] Z. Wang, X. Xiang, J. Yang dan S. Y. , "Composite Astar and B-spline algorithm for path," *International Conference on Underwater System Technology,* 2017.

[14] Wagner, Petr; Kotzian, Jiri; Kordas, Jan; Michna, Viktor, "Path planning and tracking for robots based on cubic hermite splines in real-time," *Conference on Emerging Technologies & Factory Automation (ETFA 2010),* 2010.

[15] D. Pratama, F. Ardilla, E. H. Binugroho and D. Pramadihanto, "Tilt set-point correction system for balancing robot using PID controller," 2015 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC), Bandung, Indonesia, 2015, pp. 129-135, doi: 10.1109/ICCEREC.2015.7337031